

Support desk on Corteza

Vasil Filkin · Corteza 2024.9.9 · Vue 3 Composition API · local Docker
Repo: <https://github.com/filkin1912/plexys-homework>

1. Architecture

One Corteza server process serves the API, the OAuth2 authorization server, the first-party webapps (Admin, Compose), and this homework SPA. PostgreSQL 15 holds Compose records and system data. The SPA is not a second origin.

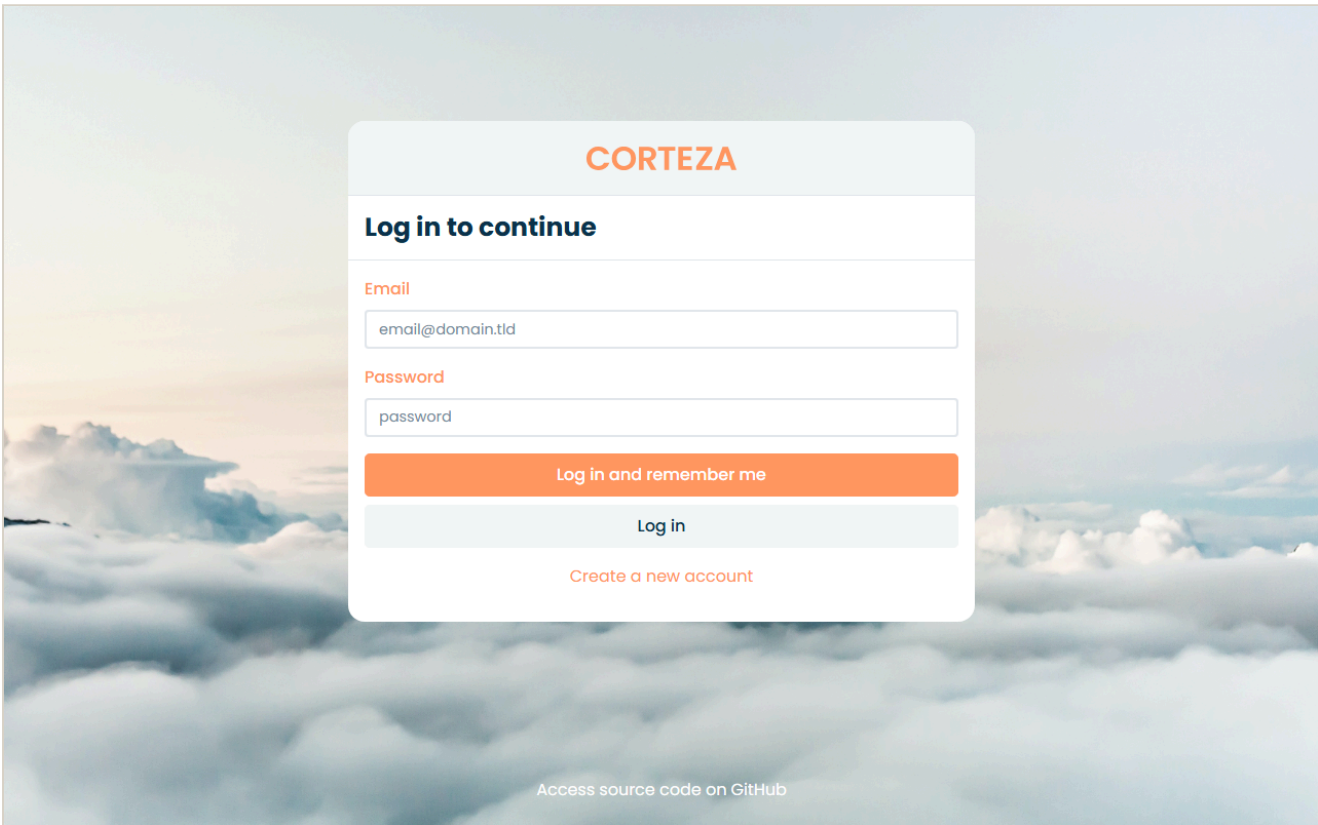
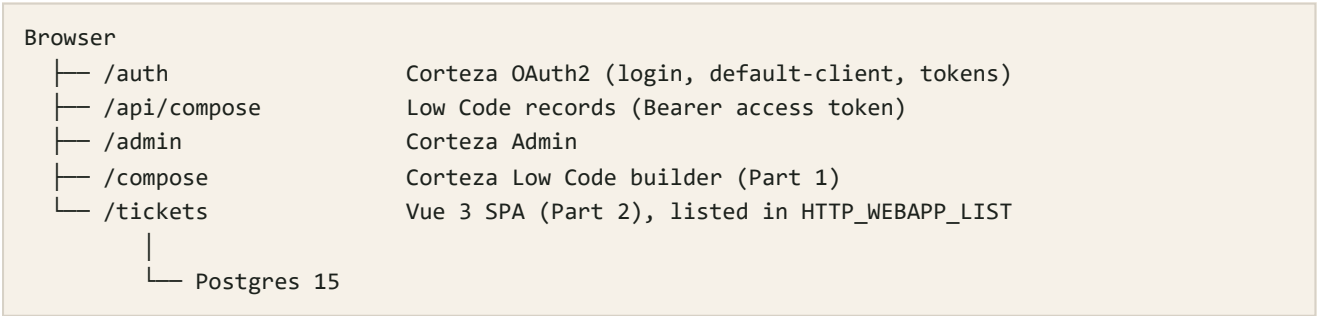


Figure 1. First-run / daily login on the local instance (Corteza 2024.9.9). The SPA at /tickets uses this same session via the default OAuth2 client.



Requests from the desk carry the signed-in user’s access token. Corteza stamps ownedBy, createdBy, createdAt and updatedAt. The SPA never writes those fields.

2. Decisions

Corteza 2024.9.9

The published docs currently live under the 2024.9 series. 2024.9.9 is the latest stable tag on Docker Hub for that series. I pinned the patch (not 2024.9 floating, not 2024.9.10-dev.*) so a clone tomorrow still boots the same bits.

How the SPA is served

Corteza can host custom webapps next to Compose: `HTTP_WEBAPP_ENABLED=true` and `HTTP_WEBAPP_LIST=...,tickets`, with the built files mounted at `/corteza/webapp/tickets` (in the 2024.9.9 image the apps live under `webapp/`, not `webapp/public/`). That is the path I used.

I did not run the SPA as a standalone Vite origin. A second origin would need a dedicated Auth Client, CORS, and a different redirect URI. First-party apps avoid that by sitting on the same host and using `/auth/oauth2/default-client`. Serving `/tickets` from Corteza keeps identity, cookies and audit trail on the platform's terms.

I also did not build the desk as Compose pages. Compose already lists records; the brief asks for a custom SPA against the module, so Compose stays the modeller and the SPA stays the operator UI.

Vue 3 instead of corteza-vue

The brief asks for Vue 3 and the Composition API. `@cortezaproject/corteza-vue@2024.9.9` depends on Vue 2.7 and bootstrap-vue. Dropping it into Vue 3 would mean rewriting its plugin surface anyway. I used `@cortezaproject/corteza-js apiClients.Compose` for record calls, and a small Composition API wrapper around the same default-client flow implemented in `corteza-vue`'s auth plugin (access token in memory, refresh token in `sessionStorage`, callback at `/tickets/auth/callback`).

UI toolkit

PrimeVue supplies the modal and confirm dialog (focus trap, Escape, labelled headers). The list is a semantic HTML table, not a card grid: ticket work is comparison across columns. Keyboard: native controls, dialogs, visible buttons — no hover-only actions.

Desk UI after the last polish

Tickets and Customers share one four-column toolbar (count · optional filters · create button) so switching tabs does not move the table or the create button. The header is one line: **Support desk** plus **Plexys homework**, nav in the middle, and **Logged-in user**: with the account email on the right.

Tickets add a search that matches *subject or customer name*, a priority dropdown (All / Low / Medium / High / Urgent), and a 7-row pager. Empty rows keep the table height so Previous / 1 2 3 / Next stay put on the last page. Status and priority use the same pill size. Priority colours:

Priority	Colour
Low	Green
Medium	Blue
High	Orange

Urgent	Red
--------	-----

These filters and the pager are extra UX. The brief asked for list / create / edit / delete of Support Ticket records as the signed-in user; that path is unchanged.

3. Data model (Part 1)

Namespace **Plexys Homework** (plexys_homework). System fields were not re-created.

Module	Field	Type	Required
Support Ticket	Subject	String	Yes
Support Ticket	Description	String, multi-line	No
Support Ticket	Status	Select: New, In Progress, Resolved, Closed	Yes
Support Ticket	Priority	Select: Low, Medium, High, Urgent	Yes
Support Ticket	Due Date	DateTime	No
Support Ticket	Customer	Record → Customer	No (bonus)
Customer	Name	String	Yes
Customer	Company	String	No
Customer	Email	Email	No

Click path in Compose: Low Code → Create namespace → Module “Customer” → Module “Support Ticket” → field Customer, type Record, related module Customer, label field Name → save → add a record of each to prove the module writes.

compose/*.json plus scripts/provision.mjs replay the same model through the Compose API if a reviewer prefers not to click. The UI path is the one the brief asked for.

4. User guide

1. Open <http://localhost:18080/tickets>. Corteza’s login appears if you have no session.
2. Sign in as the same user you created at first-run. The desk shows **Logged-in user:** and that account’s email top-right. New tickets are owned by that user.
3. **Tickets** lists subject, status, priority (colour-coded), related customer, due date, and Corteza’s updatedAt. Seven rows per page; the table height stays fixed so the pager does not jump.
4. Filter by subject or customer name, and/or by priority. Both filters apply together, then pagination covers the match set.
5. **New ticket** opens a modal. Subject, status and priority are required. Pick a customer if one exists.
6. **Edit** reopens the modal on that record. **Delete** asks for confirmation.
7. **Customers** is the related module CRUD. The create button sits in the same top-right slot as New ticket. Create a customer before linking it on a ticket.

8. Sign out returns you to Corteza Auth and drops the refresh token.

5. Admin guide

Instance

```
cd corteza
copy .env.example .env    # already done in this clone
docker compose up -d
# wait until http://localhost:18080/version returns 2024.9.9
```

First boot: open <http://localhost:18080/auth/signup>. The first account is an administrator. SMTP is off, so the email is treated as confirmed.

Local demo account used while building:

email homework@plexys.local · password Homework!2026

URLs: <http://localhost:18080/tickets> · Compose `/compose` · Admin `/admin`

GitHub: <https://github.com/filkin1912/plexys-homework>

SPA build

```
cd spa
npm install
npm run build
# docker compose already mounts spa/dist at /corteza/webapp/tickets
# refresh http://localhost:18080/tickets
```

Auth client

None was added. The SPA uses Corteza's default OAuth2 client, same as Compose. Redirect URI is <http://localhost:18080/tickets/auth/callback>. Scopes: `profile api`.

Optional API replay

```
set CORTEZA_JWT=eyJ...    # copy from an authenticated request
node scripts/provision.mjs
```

6. Limitations and next

- The API load caps at 200 records; the desk paginates 7 per page in the browser. Next would be Corteza page cursors.
- No custom RBAC beyond "signed-in user"; next would be a Support Agent role on the namespace.
- Deleting a customer does not cascade; tickets keep the old record ID until edited.
- The JS bundle is large (corteza-js + PrimeVue). Next: lazy-load PrimeVue and the customers route.

- No automated browser tests. Next: Playwright against the default-client callback.
- If the 4–5 h budget had more room: a Service module, a status filter, and a Compose record page that deep-links into the desk.

Observed, not assumed: first-run signup without SMTP marks the user confirmed; /version reports 2024.9.9; the default-client exchange is POST /auth/oauth2/default-client with application/x-www-form-urlencoded; Compose record create accepts values: [{name, value}] and fills owner/timestamps itself. Assumed: SPA fallback for /tickets/auth/callback (same as other Corteza webapps); Select option JSON shape {value, text} matches what Compose stores.